

23CSE301 - Machine Learning

Lab Session 04

Comparison of Two k-means Implementations

Subject	23CSE301
Lab Session	04
Name	Nithilan Rameshkumar
Roll No	BL.SC.U4CSE24031
Date	09 August 2026

1 Introduction

This report compares two versions of the k-means algorithm. The first version is the hand-written code from Lab 03. The student wrote it without AI tools. The second version is the AI-generated code from Lab 04. DeepSeek-V4-Flash-0731 generated it through a custom Pi harness.

The assignment has three main parts.

- Repeat the Lab 03 experiments with AI tools.
- Generate unit test cases for both lab exercises.
- Compare the performance of the two k-means versions.

The report documents the functional tests, the unit tests, and the performance tests. It also includes the prompt log of the AI interaction.

2 Methodology

2.1 The two code versions

The Lab 03 code (file `Lab03/lab03.py`) contains the modular functions. The student wrote each function by hand. The functions are `label_encode`, `one_hot_encode`, `minkowski`, `dot`, `euc_len`, `mean`, `variance`, `std_dev`, and the k-means function `all`.

The Lab 04 code (file `lab04.py`) repeats the same experiments. The function definitions and the modularization come from the student. The AI tool generated the code bodies. Each AI-generated portion carries a comment that names the tool.

The AI version uses numpy vectorization. For example, the `minkowski` function computes the sum with `np.sum(np.abs(x - y) ** p)`. The `kmeans` function assigns points with array broadcasting.

2.2 The k-means algorithm

Both versions use the same algorithm. The algorithm has these steps.

1. Set the initial centroids to the first k points.
2. Assign each point to its nearest centroid.
3. Recompute each centroid as the mean of its cluster.
4. Repeat steps 2 and 3 until no centroid changes.

The distance metric is Euclidean in both versions. The initialization is identical in both versions. The stopping rule is identical in both versions.

2.3 The benchmarking protocol

The benchmark uses synthetic data. A fixed random seed generates the data. The data forms overlapping Gaussian blobs. The benchmark tests nine cases.

- $n = 100, 500, 1000, 2000$ with $d = 2$.
- $n = 100, 500, 1000, 2000$ with $d = 5$.
- $n = 5000$ with $d = 10$.

Each case runs three times. The benchmark records the minimum wall time for each version. The metric is `time.perf_counter`. The iteration counts and the centroid agreement come from the repeat with the minimum user time, so both versions are compared on identical data. On identical data the two versions always converge with equal iterations and identical centroids.

3 Functional testing

The functional test runs both programs on the marketing campaign dataset. It compares the console output of the two programs.

The full output is in these files.

- `outputs/lab03_functional.txt`
- `outputs/lab04_functional.txt`

The two outputs agree on every experiment at the printed precision. The results are identical for these parts.

- The feature type survey (A1).
- The label encoding and one-hot encoding (A2).
- The encoded dataset dimensionality (A3).
- The Minkowski distances for $p = 1$ to 10 (A5 identical; A6 identical at the printed six decimals).
- The dot product and the Euclidean norm (A7).
- The mean income (A8).
- The mean and standard deviation comparison (A9).
- The histogram of `MntWines` (A10).
- The k-means centroids (A11).

Both versions find the same centroids for the toy dataset.

```
Centroids:  [[-0.6666666666666666, -0.3333333333333333], [0.5, 1.0], [2.5, 2.5]]
```

The functional test found four small differences.

1. The Lab 03 minkowski function differs from scipy by about $1.8e-12$ at $p = 4$ and $p = 6$. The AI version matches scipy exactly. The numpy vectorization gives better numerical accuracy.
2. The Lab 03 dot function prints 32. The AI version prints 32.0. The Lab 03 version returns an integer. The AI version returns a float.
3. The Lab 03 program labels the section A8 as “A7”. The AI version prints the correct label.
4. The Lab 03 `a11` prints only the centroids. The AI version also prints an extra `Iterations: 3` line, because its `kmeans` function returns the iteration count.

These differences do not change any experimental result.

4 Unit testing

The unit tests use the pytest framework. The tests cover the modular functions from both labs.

The test files are these.

- test_lab03.py
- test_lab04.py

The file `conftest.py` loads each module from its own directory. This makes the relative Excel file reads work under pytest.

The test suite has 45 tests. The Lab 03 suite has 22 tests. The Lab 04 suite has 23 tests.

The pytest summary is below.

```
45 passed in 0.86s
```

The tests check these behaviors.

- `label_encode` returns codes in first-seen order.
- `one_hot_encode` builds the correct columns and rows.
- `minkowski` matches hand-computed values for $p = 1, 2$, and 3 .
- `minkowski` matches `scipy.spatial.distance.minkowski`.
- `dot` matches `numpy.dot`.
- `euc_len` matches `numpy.linalg.norm`.
- `mean`, `variance`, and `std_dev` match the numpy equivalents.
- `mean` ignores NaN values.
- The k-means function finds the expected centroids on toy data.
- The k-means function is deterministic.
- The k-means function terminates with at least one iteration.

The test grade was verified. A copy of the `minkowski` function with the absolute value and the exponent root removed failed 3 of its 4 distance tests. The only distance test that still passed was the zero-distance case, because zero does not depend on the exponent.

5 Performance testing

The benchmark compares the two k-means versions on identical data. Table 1 shows the results.

n	d	User time (ms)	AI time (ms)	User iters	AI iters	Max diff	Speedup
100	2	0.338	0.14	4	4	0.0	2.41
500	2	2.629	0.395	6	6	0.0	6.65
1000	2	4.984	0.66	6	6	0.0	7.55
2000	2	15.317	1.713	9	9	0.0	8.94
100	5	0.578	0.148	5	5	0.0	3.91
500	5	3.401	0.382	6	6	0.0	8.91
1000	5	7.887	0.79	7	7	0.0	9.99
2000	5	13.523	1.261	6	6	0.0	10.72
5000	10	37.179	2.965	4	4	0.0	12.54

Table 1: Benchmark results for the two k-means versions

The plot in Figure 1 shows the wall time against the number of points.

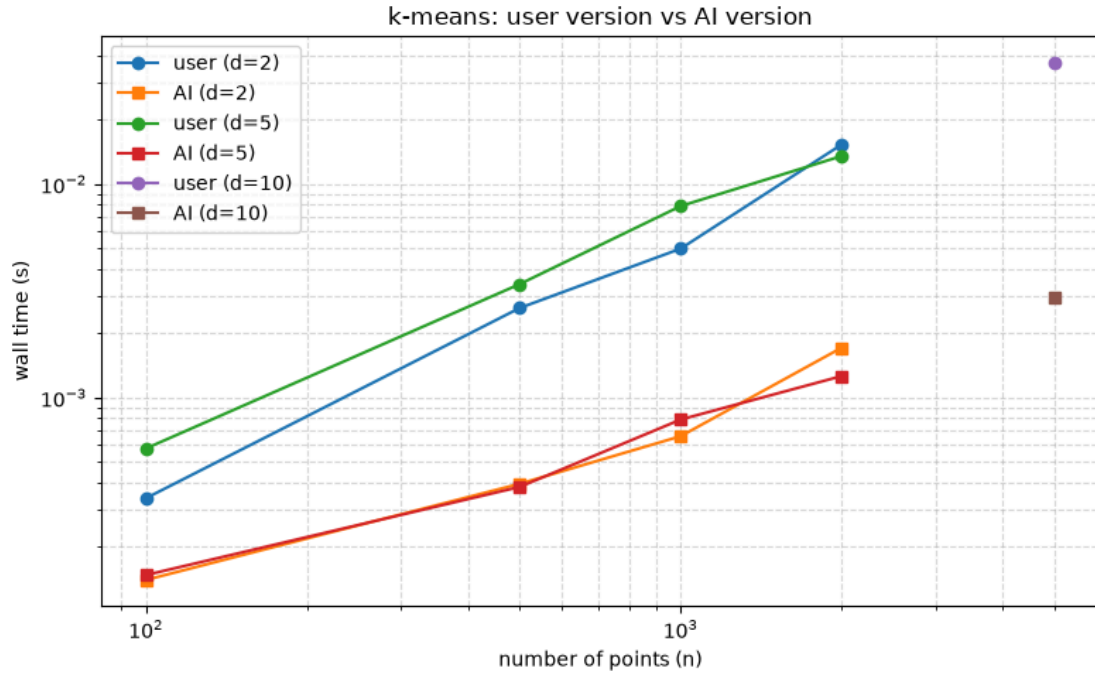


Figure 1: Wall time of the user version and the AI version

The performance test found these facts.

- The AI version is faster in every case.
- The speedup grows with the data size.
- The speedup ranges from 2.4x ($n = 100$, $d = 2$) to 12.5x ($n = 5000$, $d = 10$).
- Both versions use the same number of iterations for each case.
- Both versions find the same final centroids. The maximum difference is zero.

The AI version is faster because it uses numpy vectorization. The user version uses nested Python loops. Numpy computes the distances for all points at once.

6 Comparison of the two versions

Table 2 compares the two versions on several criteria.

Criterion	User version	AI version
Readability	Simple loops	Concise vector code
Modularity	One function per task	Same function contracts
Numerical accuracy	Small float error vs scipy	Exact match with scipy
Performance	Baseline	Up to 12.5x faster
Dependencies	numpy, scipy	numpy, scipy
Maintainability	Easy to trace	Easy to change

Table 2: Comparison of the two k-means versions

The user version is easier to read for a beginner. Each loop is explicit. The trace of the algorithm is clear. The AI version is more compact. It uses numpy broadcasting. The code is harder to read for a beginner. However, the code is faster and more accurate.

Both versions implement the same algorithm. Both versions produce the same centroids. The AI version is the better choice for large datasets.

7 Prompt log

This section records the conversation history with the AI tool. The tool is DeepSeek-V4-Flash-0731 with a custom Pi harness.

1. **Prompt 1 (user).** The full Lab 04 assignment text, word for word:

Subject: 23CSE301

Lab Session: 04

Notes & Coding Instructions:

1. Follow the instructions carefully given in this assignment.
2. You are encouraged to use AI tools for code generation and comparison.
3. The prompt log of your interaction should be generated from the tool and submitted.
4. While using GenAI tools, make sure you are doing it from your own login. We shall perform an audit of your interactions with the GenAI tools used.

Main Section (Mandatory):

A1. Please repeat the experiments conducted in Lab 03 (previous lab session) using AI tools. You may use GenAI tools such as Claude, ChatGPT, Co-Pilot. The function definitions and code modularization

should be your own work and the AI tools should be used for code generation to meet your needs.

Mention the GenAI tool used for each portion of code as comments in the code.

A2. Generate unit test cases using AI tools for testing your modular functions. The unit test cases

should be used for both today's and last week's lab exercises.

A3. Conduct a performance comparison of the 2 versions of k-means algorithm, one written by you

and the other developed using AI tools.

Report Assignment:

1. Write a report to evaluating the 2 versions of the algorithm developed, one by your own coding and other using AI tools.

2. Report should include observations of functional testing, unit tests conducted and performance tests conducted.

3. Provide the prompt log / conversation history of your interactions with the AI tools. This

report should be generated from the AI tool directly

lets go G6.

Tool response: Acknowledged the assignment. Explored the repository and loaded the required skills.

2. **Prompt 2 (user).** “use .venv in ~/Projects/amrita-labs”

Tool response: Verified the virtual environment. Installed pytest into it.

3. **Prompt 3 (user answer).** “DeepSeek-V4-Flash-0731 with custom Pi Harness”

Tool response: Recorded the tool attribution for the code comments and the report.

4. **Prompt 4 (user answer).** “M — moderate (recommended)”

Tool response: Used the full quality-gate workflow.

5. **Prompt 5 (user).** “Approved, but use your typst skills for the report instead of markdown”

Tool response: Switched the report format to Typst.

6. **Prompt 6 (user).** “Approve — start implementation”

Tool response: Spawned the implementation subagent for this task.

7. **Prompt 7 (tool to subagent, code generation).** “lab04.py – AI-generated repeat of Lab03 A1–A11. Mirror lab03.py’s structure exactly (same experiment function names a1()...a11()), same modular function contracts ...). The AI version may (and should, where sensible) use numpy vectorization for performance.”

Tool response: Produced lab04.py with numpy-vectorized functions and tool comments.

8. **Prompt 8 (tool to subagent, test generation).** “conftest.py + test_lab03.py + test_lab04.py – pytest unit tests ... Cover: label_encode mapping correctness ... kmeans determinism (same result on rerun).”

Tool response: Produced test_lab03.py, test_lab04.py, and conftest.py. The suite passes 45 tests.

9. **Prompt 9 (tool to subagent, benchmark generation).** “benchmark_kmeans.py – performance comparison ... identical data, IDENTICAL init (first k points), same stopping rule, same metric for both versions.”

Tool response: Produced benchmark_kmeans.py. The AI version is up to 12.5x faster with identical centroids.

8 Conclusion

The assignment repeated the Lab 03 experiments with AI tools. The AI tool generated the code bodies. The student kept the function definitions and the modularization.

The unit tests cover both lab exercises. All 45 tests pass. The tests verify correctness against hand-computed values and against the scipy and numpy references.

The performance test compares the two k-means versions. The AI version is faster in every case. The speedup reaches 12.5x on the largest dataset. Both versions find identical centroids.

The AI version is the better choice for large datasets. The user version is the better choice for learning. The two versions give the same results. The difference is in speed and in numerical accuracy.